

《软件工程》实验报告

实验三：《AI Agent 系统》设计规格说明

专业班级：人工智能 2301

学生学号：542307250114

542307250113

542307250110

542307250112

学生姓名：刘勇泽、刘洋、李容昊、梁家诚

指导教师：夏永泉、支俊

目录

1 概述

1.1 编写目的

1.2 项目背景

1.3 参考资料

1.4 术语与缩写

2 需求概述

2.1 系统目标

2.2 功能需求

2.3 用户角色与使用场景

2.4 数据需求

2.5 性能与约束需求

2.6 非功能需求

3 结构设计

3.1 总体设计

3.1.1 系统结构图

3.2 功能分配

3.2.1 用户认证与授权

3.2.2 会话管理

3.2.3 流式对话与工具调用

3.2.4 工具统计与系统诊断

3.2.5 发布报告与导出

3.3 接口设计

3.3.1 外部接口设计

3.3.2 内部接口设计

3.4 数据结构设计

3.4.1 核心数据结构

3.4.2 数据库设计

3.4.3 数据结构与程序关系

3.5 出错处理设计

3.6 UML 图与流程说明

4 构件（过程）设计

4.1 数据库与持久层详细设计

4.2 业务逻辑层详细设计

- 4.3 用户界面层详细设计
- 4.4 测试与验证

四设计说明书分工

四设计说明书分工

成员	负责内容
刘勇泽	系统总体设计、后端核心接口、认证与报表说明统稿
刘洋	Web 前端展示、交互流程与文档排版
李容昊	CLI 交互、联调验证与冒烟测试
梁家诚	UML 图整理、部署脚本与运行证据归档

1 概述

1.1 编写目的

本文档用于描述 AI Agent 系统的总体功能、结构设计与过程设计，作为程序编写、系统测试、部署验收与后续维护的依据。

- (1) 明确系统边界、功能边界与模块职责。
- (2) 统一开发成员对接口契约、数据结构与业务流程的理解。
- (3) 为后续编码、联调、测试和验收提供可追踪依据。
- (4) 把“能跑起来”的实现整理成符合软件工程规范的设计说明书。

1.2 项目背景

本项目面向 AI + Java 技术融合场景，实现一个可运行的 AI Agent 系统。系统围绕“用户认证、会话管理、流式对话、工具调用、统计报表与多端协同”展开，强调工程化而非单纯的模型调用。

项目采用前后端分离架构，后端以 Spring Boot 为核心，前端提供 Web 交互界面，CLI 提供命令行能力，并通过 Docker 和脚本完成部署与冒烟验证。

1.3 参考资料

- (1) 《软件工程》实验三格式要求。
- (2) AI Agent 项目源码与 README.md。
- (3) Code_Wiki.md 中的系统结构与接口说明。
- (4) 后端、前端、CLI 与脚本目录中的实际实现。
- (5) Spring Boot、React、JWT、SSE 与 Docker 相关官方资料。

1.4 术语与缩写

术语	含义
AI Agent	可调用工具、可维护会话上下文的智能代理系统
JWT	无状态身份认证令牌
SSE	Server-Sent Events，服务端推送流式事件
Session	用户一次完整的聊天会话
Tool Trace	工具调用审计与执行轨迹
CLI	命令行交互客户端
OpenAI Compatible	兼容 OpenAI 请求格式的模型提供方

2 需求概述

2.1 系统目标

系统目标不是单纯提供聊天窗口，而是形成一套具备工程可用性的 AI Agent 平台：支持多端访问、会话持久化、流式输出、工具审计、报表导出和运行验证。

- (1) 支持用户注册、登录、刷新令牌与个人信息查询。
- (2) 支持创建会话、查询会话、导出会话消息。
- (3) 支持普通对话与 SSE 流式对话。
- (4) 支持模型路由、工具调用与审计记录。
- (5) 支持系统健康检查、工具统计和发布报告导出。
- (6) 支持 Web 端和 CLI 端协同使用，并可通过脚本完成冒烟验证。

2.2 功能需求

- (1) 认证授权：用户必须登录后访问受保护接口，登录态通过 JWT 维护。
- (2) 会话管理：每个用户可创建多个会话，消息与会话绑定存储。
- (3) 智能对话：用户输入消息后，系统调用模型服务生成回复。
- (4) 工具调用：模型可在回答过程中调用代码搜索、文件读取、目录浏览、POM 分析等工具。
- (5) 统计报表：系统可按时间窗口统计工具调用情况，并生成发布报告。
- (6) 系统诊断：系统可检查数据库、缓存与模型服务可用性。
- (7) 多端协同：Web 端负责交互界面，CLI 端提供开发者命令行入口。

2.3 用户角色与使用场景

角色	典型场景
普通用户	在 Web 或 CLI 端登录后进行对话、查看历史会话、导出结果
开发维护人员	检查服务状态、查看报表、执行部署与冒烟验证
系统管理员	关注认证、限流、健康检查和运行统计

2.4 数据需求

- (1) 用户数据：账号、密码哈希、令牌版本、创建时间。
- (2) 会话数据：会话 ID、标题、模型提供方、模型名称、所属用户。
- (3) 消息数据：消息角色、内容、工具调用信息、创建时间。
- (4) 工具审计数据：工具名、参数、执行状态、耗时、输出摘要。
- (5) 报表数据：健康检查结果、模型列表、统计指标、生成时间。

2.5 性能与约束需求

- (1) 流式连接最长 300 秒，心跳间隔 10 秒，防止长连接失活。
- (2) Agent 最大工具循环步数为 4，避免模型无限调用工具。
- (3) 登录与聊天接口应具备限流能力，降低滥用风险。
- (4) 系统运行依赖 PostgreSQL、Redis 与 OpenAI 兼容模型服务。
- (5) 路径访问与文件读取必须限制在工作区内，防止越权读取。

2.6 非功能需求

- (1) 安全性：采用 JWT、密码加密、接口鉴权、速率限制和安全路径解析。
- (2) 可维护性：控制器、服务、仓储、工具层分层明确，便于扩展。
- (3) 可扩展性：模型提供方和工具集可按接口扩展。
- (4) 可观测性：请求上下文、会话 ID 与用户 ID 可追踪，工具调用可统计。
- (5) 可部署性：支持 Docker Compose 与脚本化部署、回滚和冒烟测试。
- (6) 可测试性：关键接口与报表生成逻辑有对应单元测试或验证脚本。

3 结构设计

3.1 总体设计

系统采用典型的分层架构：表现层负责请求入口与交互，应用层负责业务编排，基础设施层负责认证、会话、工具调用、模型适配和数据持久化。

层次	主要职责
Web/CLI 表现层	负责用户输入、结果展示、流式输出和导出操作
Controller 层	负责 REST 接口、SSE 连接、参数校验与鉴权
Service 层	负责认证、会话、Agent 循环、报表与诊断编排
Infrastructure 层	负责数据库、Redis、模型服务与外部资源访问

同步对话时，Controller 接收请求后校验身份与限流，再由 AgentService 拉取会话历史、解析模型配置并执行工具循环；流式对话时，SseEmitter 负责事件推送，心跳线程负责保持长连接，最终将回复与工具轨迹落盘。

3.1.1 系统结构图

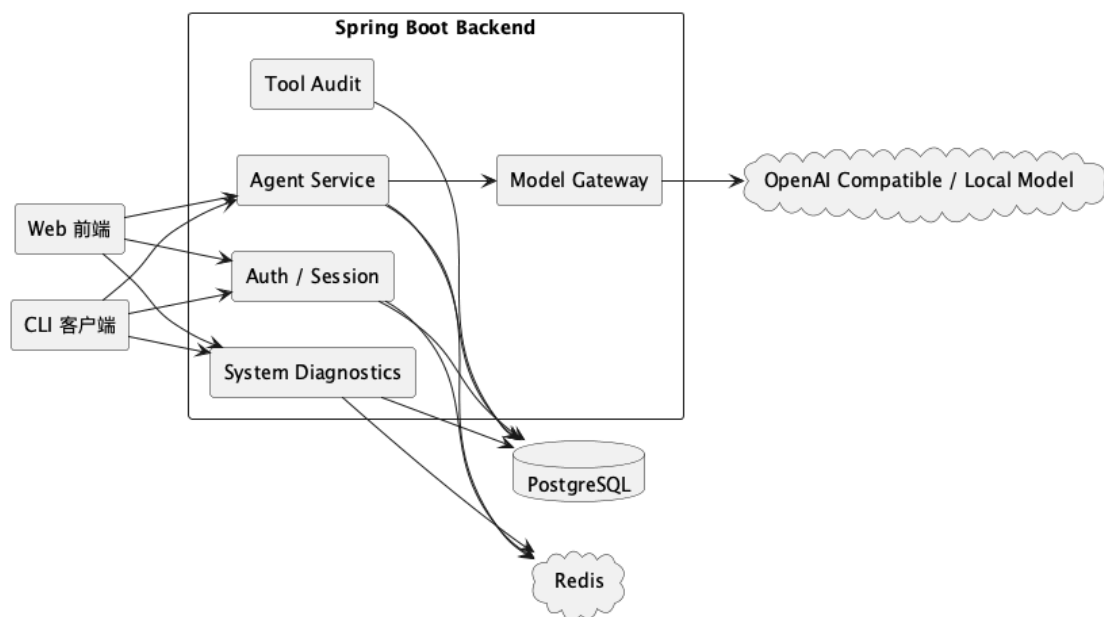


图 3-1 系统结构图

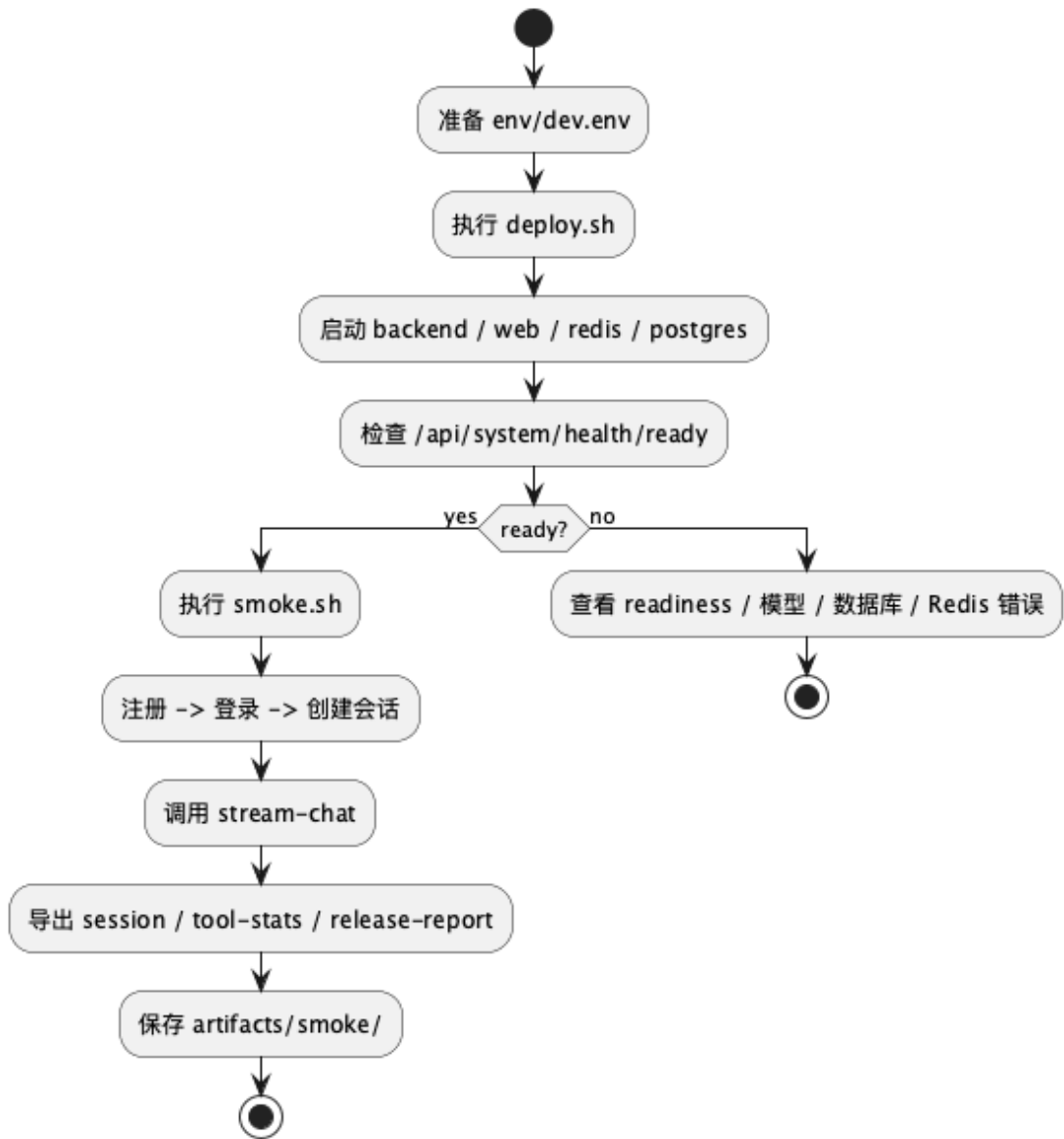


图 3-2 部署与冒烟验证流程图

3.2 功能分配

模块	主要类或接口	职责
认证授权	AuthController / AuthService / JwtService	注册、登录、刷新令牌、 获取当前用户
会话管理	SessionController / SessionService	创建会话、查询历史、 导出消息
智能对话	AgentController / AgentService / ModelGateway	同步对话、流式对话、 工具循环
工具编排	AgentToolOrchestrator / CodeToolService	搜索代码、读文件、浏 览目录、分析 POM
统计报表	ToolStatsController / ReleaseReportCon- troller	生成工具统计与发布报 告
系统诊断	SystemController / SystemDiagnosticsSer- vice	健康检查、模型列表与 可用性验证
前端交互	web/src	Web 界面、会话列表、 流式输出与导出按钮
命令行交互	cli/src/main/java	登录、对话、统计和报 表命令

功能分配遵循“一个功能块对应一组前端页面截图和一张时序图”的原则，确保实验报告中的功能描述、界面展示和过程说明一一对应。

3.2.1 用户认证与授权

登录模块负责注册、登录与刷新令牌，前端页面展示邮箱、密码与验证码输入区；后端通过 JWT 维护登录态，并写入 token_version 以支持令牌轮转。



图 3-3 用户认证与授权前端页面图

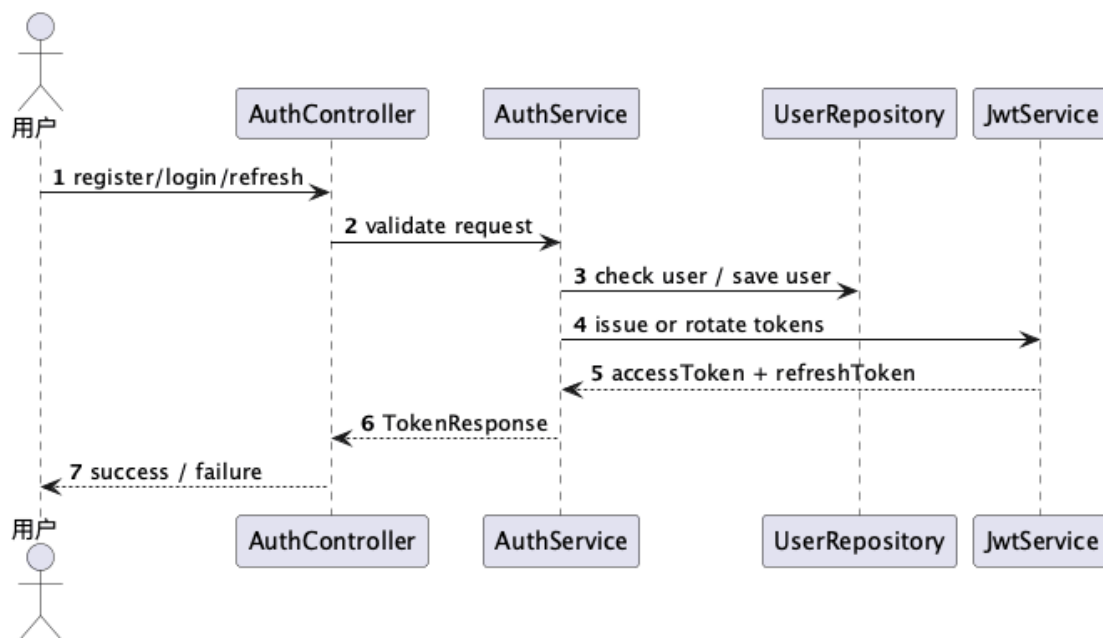


图 3-4 用户认证与授权时序图

3.2.2 会话管理

会话模块负责创建会话、切换会话、查询历史消息与导出消息；前端页面体现会话列表、新会话表单和会话切换入口，便于从工程角度说明多会话管理。

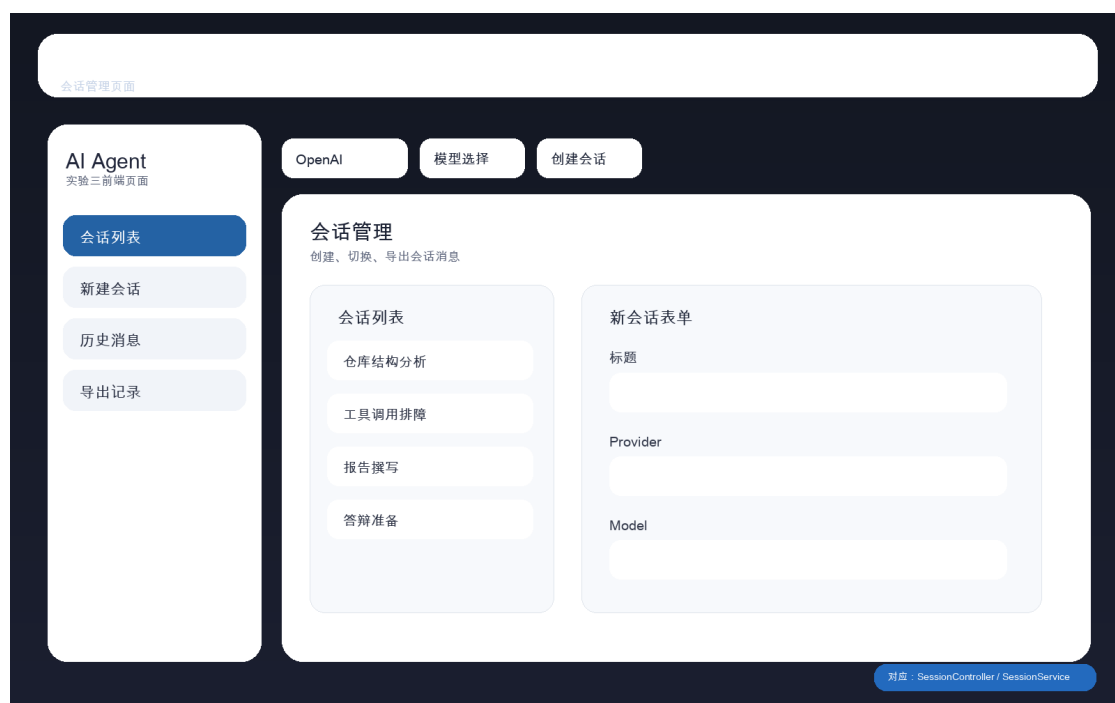


图 3-5 会话管理前端页面图

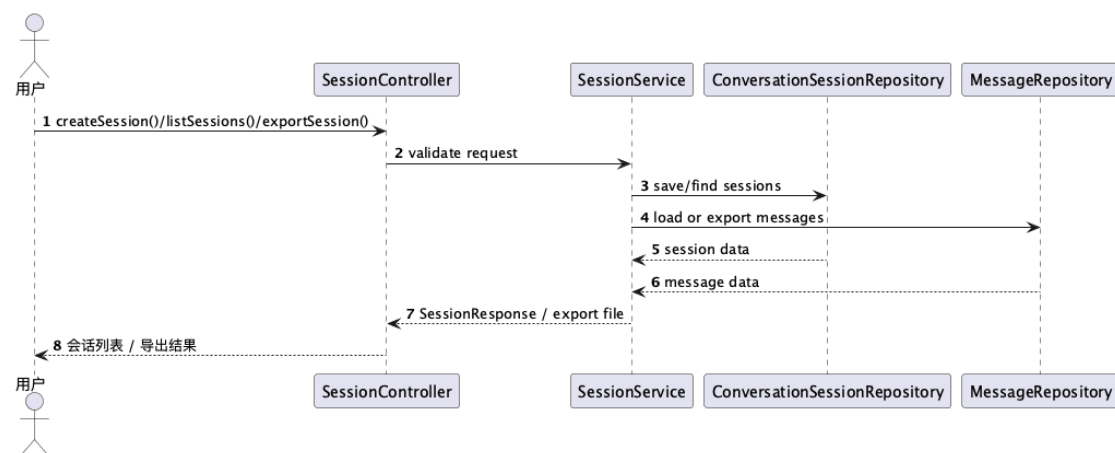


图 3-6 会话管理时序图

3.2.3 流式对话与工具调用

流式对话模块负责接收用户输入、调用模型服务、执行工具并通过 SSE 推送 chunk 与 heartbeat；前端页面展示消息气泡、输入框和发送按钮，直观看到生成过程。

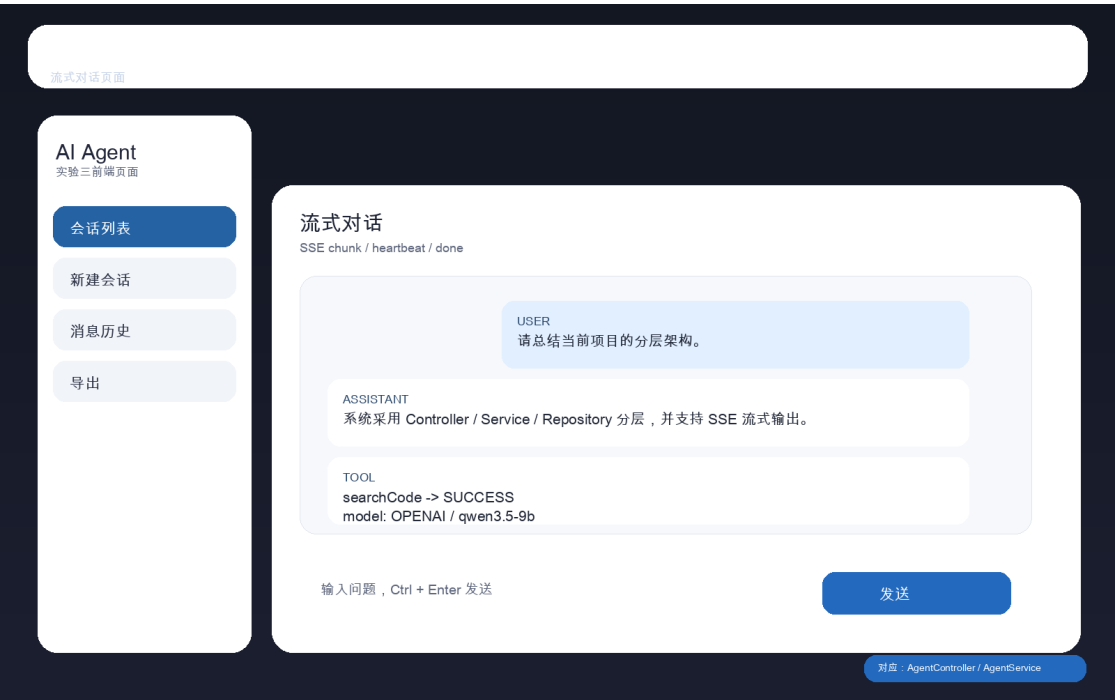


图 3-7 流式对话与工具调用前端页面图

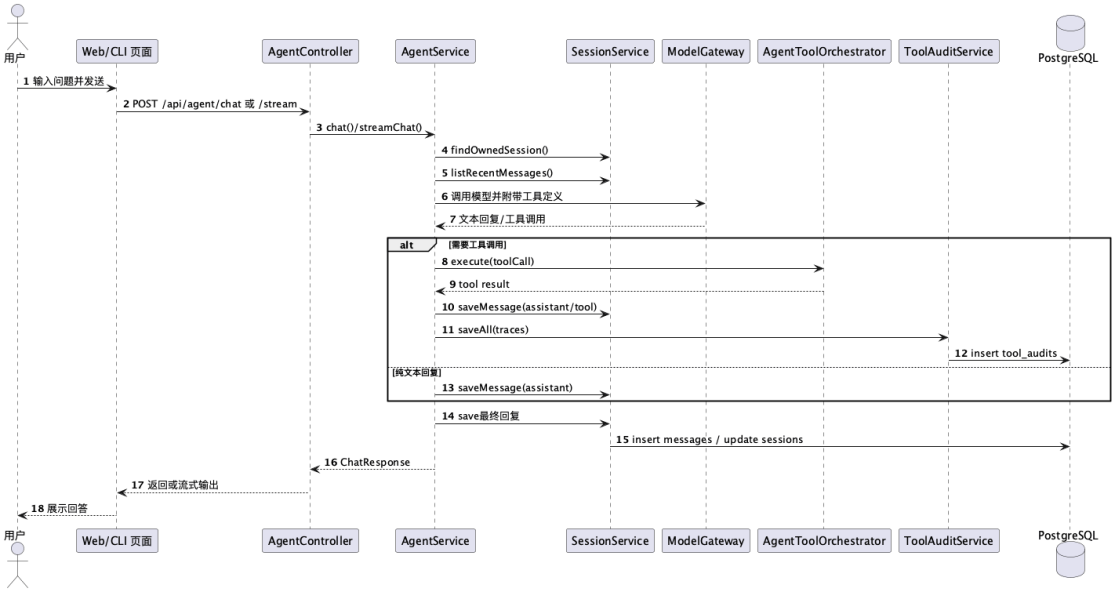


图 3-8 流式对话与工具调用时序图

3.2.4 工具统计与系统诊断

统计模块负责聚合工具调用次数、成功率与耗时分布；诊断模块负责检查数据库、Redis 与模型服务状态。前端页面以统计卡片和健康检查列表呈现系统运行情况。



图 3-9 工具统计与系统诊断前端页面图

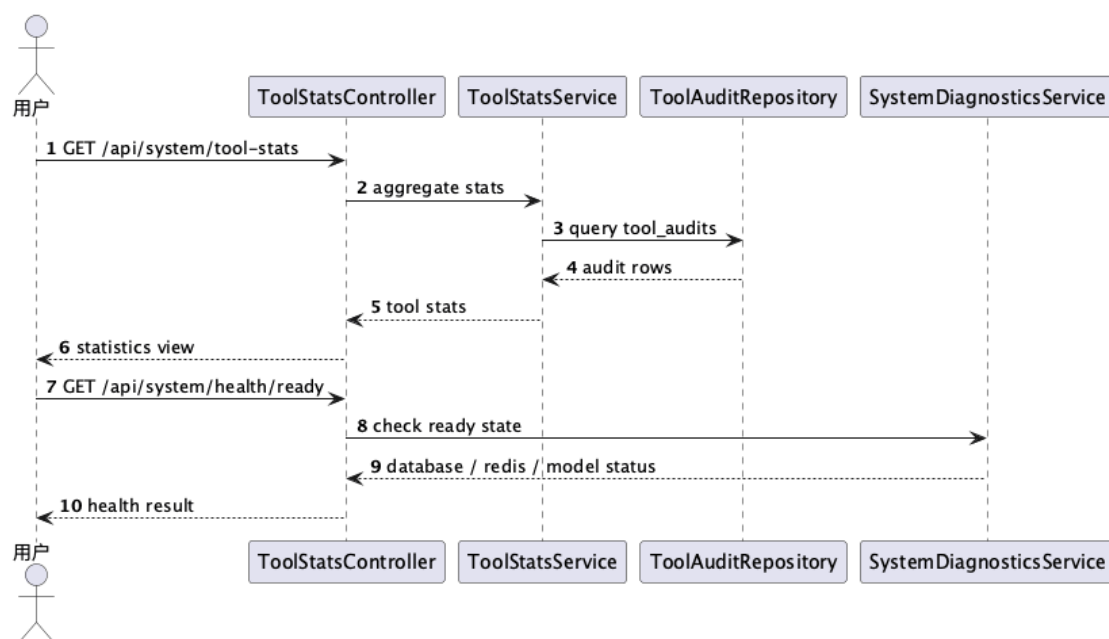


图 3-10 工具统计与系统诊断时序图

3.2.5 发布报告与导出

发布报告模块负责把健康检查、模型列表、统计指标和运行证据汇总成报告；前端页面提供导出口，体现系统从运行到归档的闭环能力。



图 3-11 发布报告与导出前端页面图

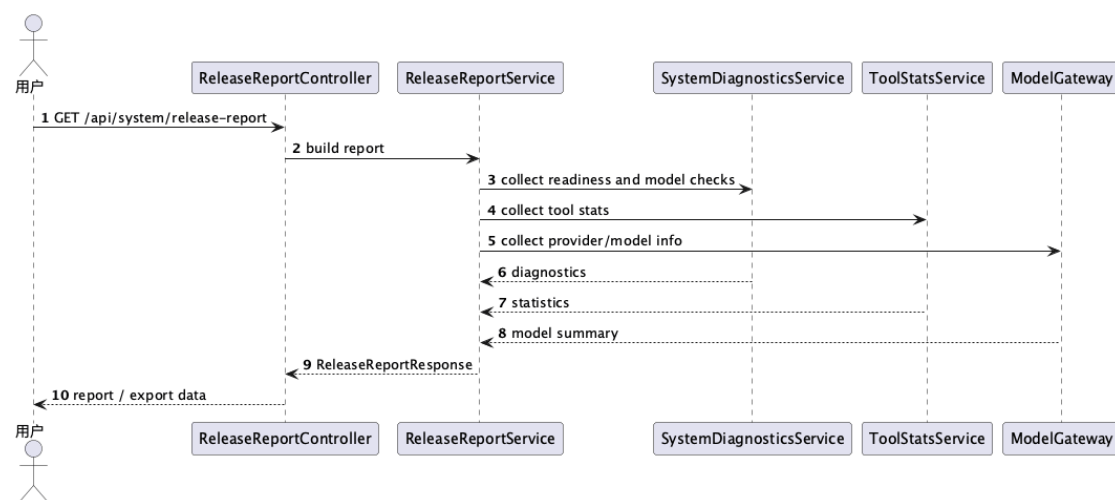


图 3-12 发布报告与导出时序图

3.3 接口设计

3.3.1 外部接口设计

接口	功能
POST /api/auth/register	用户注册
POST /api/auth/login	用户登录并生成 JWT
POST /api/auth/refresh	刷新访问令牌
POST /api/sessions	创建会话
GET /api/sessions/{id}/messages	查询会话消息
POST /api/agent/chat	同步对话
POST /api/agent/chat/stream	SSE 流式对话
GET /api/system/health/ready	系统就绪检查
GET /api/system/tool-stats	工具统计查询
GET /api/system/release-report	发布报告查询

外部接口统一采用 JSON 结构返回，认证接口需要携带账号密码或刷新令牌，受保护接口需要在 Authorization 头中携带 Bearer Token。流式接口返回 text/event-stream，事件类型包含 meta、chunk、done、heartbeat 与 error。

3.3.2 内部接口设计

- (1) Controller 只负责参数校验、上下文注入和异常转换，不直接承担业务规则。
- (2) Service 之间通过明确 DTO/Response 对象传递数据，避免直接暴露实体。
- (3) ModelGateway 负责统一模型提供方差异，便于后续接入 OpenAI 兼容模型或本地模型。
- (4) ToolOrchestrator 负责工具规范定义与调用执行，保证模型与工具解耦。
- (5) Repository 与 Redis 组件负责持久化与缓存，业务层不直接写 SQL。

3.4 数据结构设计

3.4.1 核心数据结构

结构	用途
User	保存用户账号、密码哈希和令牌版本
ConversationSession	保存会话标题、模型配置和所属用户
Message	保存用户、助手与工具消息
ToolExecutionResult	保存单次工具执行结果
ToolStatsResponse	保存工具统计结果
ReleaseReportResponse	保存发布报告内容

3.4.2 数据库设计

系统数据库采用 PostgreSQL，核心表包括 users、conversation_sessions、messages 和 tool_audits。表之间通过主外键形成稳定的数据关系，满足认证、会话、消息与审计四类业务需求。

表名	关键字段	外键 / 索引	设计说明
users	id, email, password_hash, token_version, created_at	email 唯一索引	保存用户账号、密码哈希和令牌轮转信息；token_version 用于强制注销旧令牌。
conversation_sessions	id, user_id, title, provider, model, created_at, updated_at	user_id → users.id; (user_id, updated_at) 索引	保存会话元数据、默认模型配置和所属用户，支撑会话列表与最近会话排序。

表名	关键字段	外键 / 索引	设计说明
messages	id, session_id, role, content, tool_trace, provider, model, created_at	session_id → conversation_sessions.id; (session_id, created_at) 索引	保存会话消息、工具追踪和模型元信息，构成对话历史的主要证据链。
tool_audits	id, user_id, session_id, tool_name, args_json, status, duration_ms, provider, model, created_at	user_id → users.id; session_id → conversation_sessions.id; (session_id, created_at) 索引	保存工具调用审计记录和耗时信息，便于统计成功率、延迟和失败原因。

- (1) 一个用户可以拥有多个会话。
- (2) 一个会话包含多条消息，消息按时间顺序追加。
- (3) 一个 Agent 回合可能产生多次工具调用和工具结果。
- (4) 工具审计记录和会话消息共同构成可追踪证据链。
- (5) Redis 用于会话缓存、限流窗口与运行时状态辅助存储。
- (6) 数据库迁移通过 Flyway 管理，确保字段演进可回溯。

3.4.3 数据结构与程序关系

控制器接收请求后，将 DTO 传入 Service；Service 根据 Session、Message、ToolAudit 等结构完成业务编排；Repository 与 Redis 层负责持久化和缓存，最终通过 Response DTO 对外返回。

3.5 出错处理设计

- (1) 参数校验失败时返回统一错误响应，提示具体字段问题。
- (2) 身份校验失败时返回未授权错误，并要求重新登录或刷新令牌。
- (3) 登录和聊天接口触发限时时，返回友好提示并建议稍后重试。
- (4) 模型调用或工具调用出错时，Agent 会安全停止，而不是无限重试。
- (5) 流式连接异常时，SSE 连接会被关闭，前端和 CLI 端显示故障原因。
- (6) 启动阶段通过环境检查、Redis 检查和模型可用性检查，尽量在上线前暴露问题。

3.6 UML 图与流程说明

图 3-1 展示系统分层与运行边界，说明 Web、CLI、后端、数据库、缓存和模型服务的关系。

图 3-2 展示部署后的一键冒烟验证闭环，用于说明工程交付过程。

图 3-3 至图 3-15 分别说明五个功能块的前端页面、对应时序、核心组件协作、前端页面结构和数据库结构。

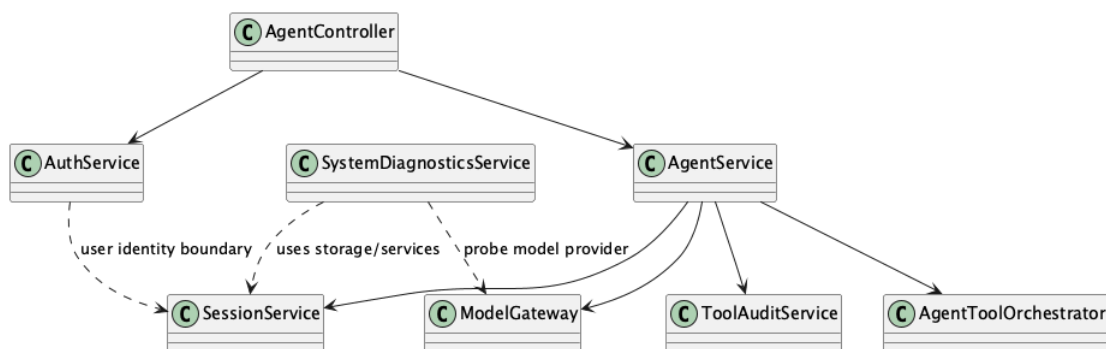


图 3-13 核心组件图

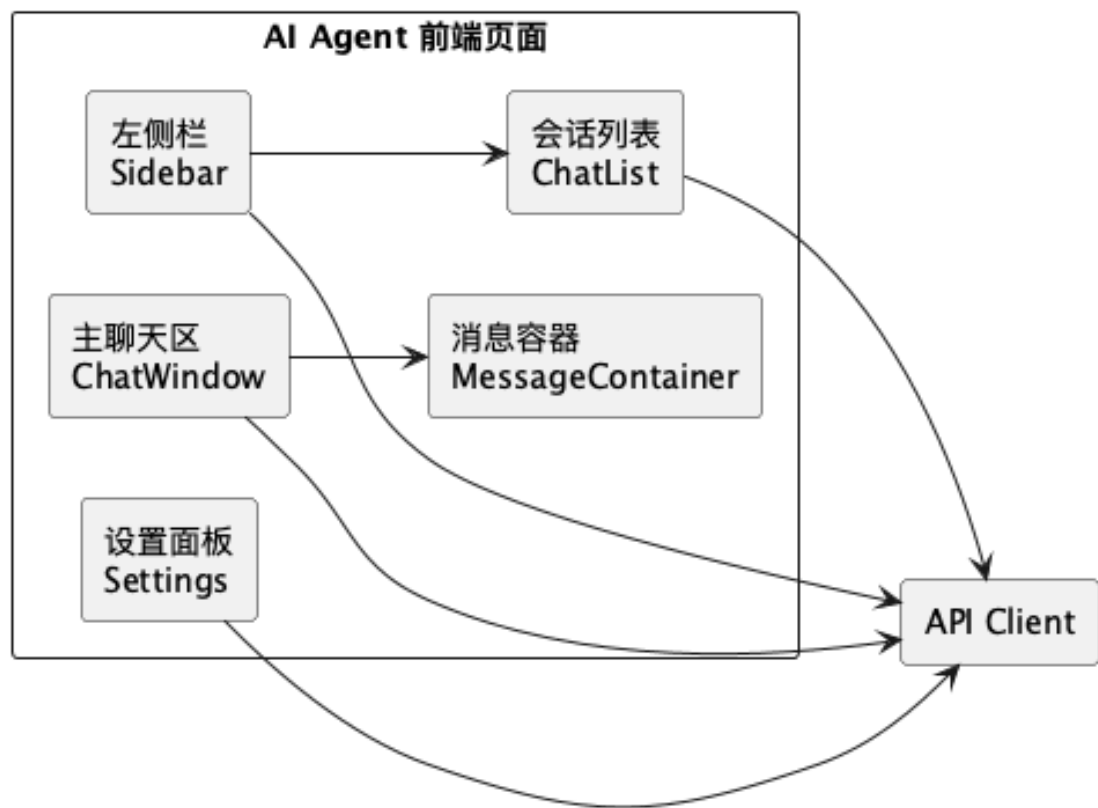


图 3-14 前端页面结构图

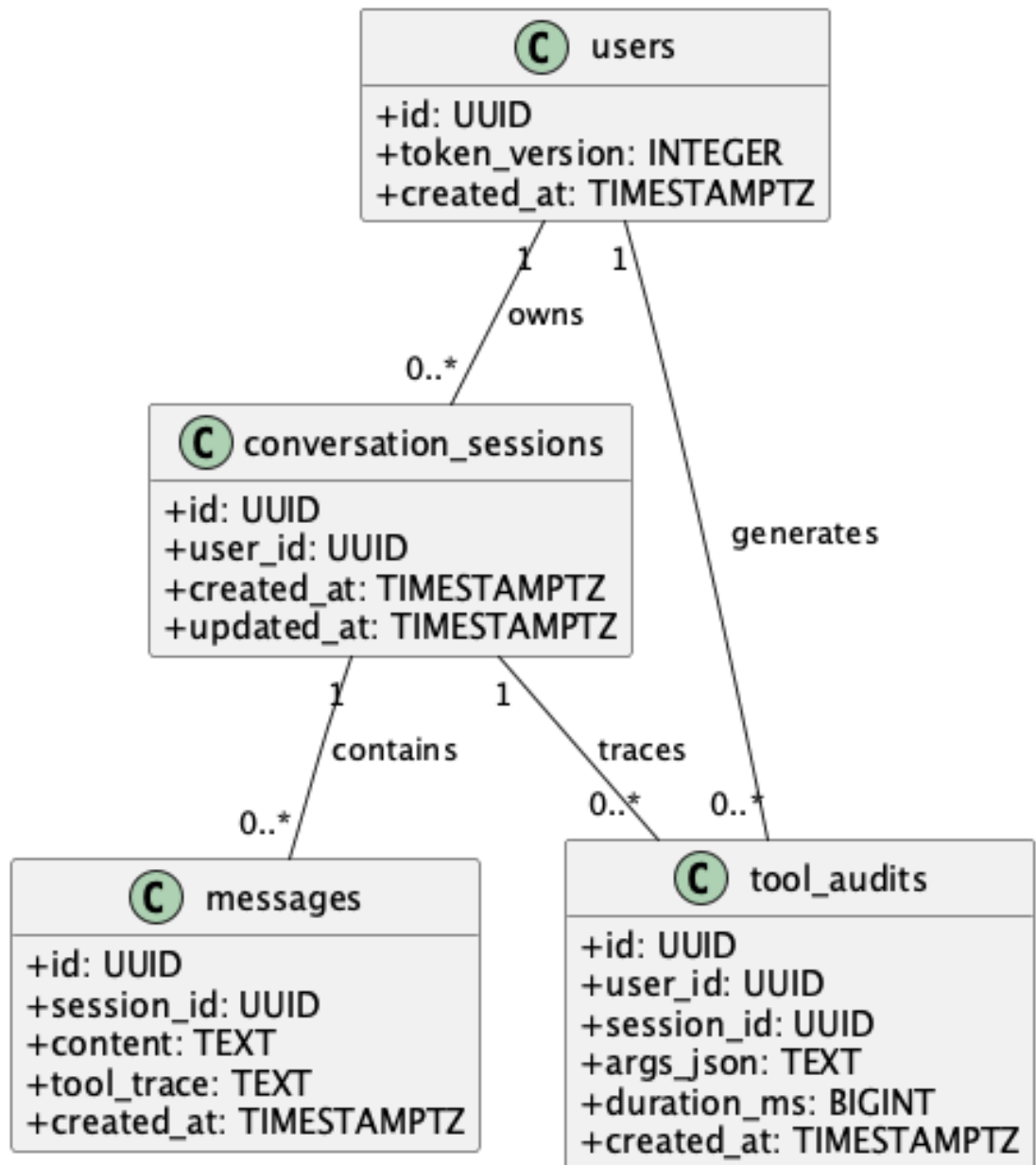


图 3-15 数据库结构图

4 构件（过程）设计

4.1 数据库与持久层详细设计

持久层负责保存用户、会话、消息和审计记录。后端采用 Repository 方式屏蔽底层存储细节，业务层只关注实体和查询语义，不直接关心 SQL 细节。

- (1) UserRepository 负责用户注册、登录查询与令牌版本控制。
- (2) ConversationSessionRepository 负责会话创建、归属校验和会话列表查询。
- (3) MessageRepository 负责会话消息追加和历史消息读取。
- (4) ToolAuditService 负责保存工具执行轨迹和聚合统计。
- (5) RedisSessionCacheService 与 RedisRateLimiterService 负责缓存和限流。

从软件工程角度看，这一层的关键价值是把数据访问与业务规则隔离，避免控制器和业务服务中出现重复的存取逻辑，从而降低耦合度。

4.2 业务逻辑层详细设计

4.2.1 认证业务流程

注册时先检查邮箱唯一性，再校验密码强度，最后对密码进行哈希后落库。

登录时先校验账号密码，再签发访问令牌和刷新令牌。

刷新令牌时检查 tokenVersion，完成轮转后再签发新令牌对。

4.2.2 会话与消息流程

创建会话时为用户保存模型提供方与模型名称，作为后续对话默认配置。

对话时先读取最近消息作为上下文，再将用户消息写入会话消息表。

导出时支持 JSON 与 Markdown 两种结果，便于审阅和归档。

4.2.3 Agent 循环算法

- (1) 收集会话历史并裁剪到上下文预算内。
- (2) 构造 system 消息、历史消息和工具定义，发送给模型服务。
- (3) 若模型只返回文本，则结束本轮循环。
- (4) 若模型发起工具调用，则执行 searchCode、readFile、listRepoTree、analyzePom 等工具。
- (5) 工具结果写回会话并记录审计；若工具失败则安全停止。
- (6) 若达到最大工具步数 4，则强制收敛，防止无限循环。

这一算法体现了软件工程中的“控制复杂度”思想：通过步数上限、错误停止和上下文预算，避免 Agent 行为失控。

4.2.4 SSE 流式与心跳流程

Controller 建立 SseEmitter，并设置 300 秒超时。

心跳调度器每 10 秒发送 heartbeat，保证长连接不被中间链路回收。

最终回复通过 chunk 事件逐步输出，结束时发送 done 事件。

若出现异常，发送 error 事件并关闭连接。

4.2.5 统计与报表流程

ToolStatsController 根据时间窗口和可选 sessionId 统计工具调用情况。

ReleaseReportService 将健康检查、模型列表和工具统计聚合成报表。

导出接口支持 Markdown 与 JSON，便于归档与人工审阅。

4.3 用户界面层详细设计

4.3.1 Web 前端

Web 端以 React + TypeScript 实现，提供登录、会话列表、消息展示和导出按钮。

侧边栏提供工具统计过滤和报表导出入口，前端会根据接口结果更新状态。

流式聊天时界面会逐步追加内容，让用户直观看到模型生成过程。

前端页面结构由 Sidebar、ChatList、ChatWindow、MessageContainer、Settings 等组件构成，适合在实验报告中作为页面设计说明展示。

4.3.2 CLI 命令行客户端

CLI 端基于 Picocli，提供 login、chat、stream-chat、tool-stats、release-report 等命令。

stream-chat 命令会打印 connecting、first token、done 等状态，便于开发调试。

tool-stats 与 release-report 命令支持 JSON/Markdown 输出，便于终端和脚本集成。

4.3.3 部署与验证流程

部署时先准备环境变量，再执行部署脚本与冒烟脚本。

smoke.sh 会自动完成注册、登录、创建会话、流式对话、导出会话、导出报表等流程。

执行过程中生成 readiness、models、stream.sse、session-export、tool-stats、release-report 等证据文件。

4.4 测试与验证

- (1) 接口测试：验证登录、会话、对话、统计与导出接口可用。
- (2) 流式测试：验证 SSE 事件顺序、心跳与完成事件正确。
- (3) 安全测试：验证鉴权失败、限流、路径越权和无效令牌处理。
- (4) 集成测试：验证 Web 端、CLI 端与后端服务协同工作。
- (5) 冒烟测试：验证一键部署后的最小闭环链路可运行。

从实验交付角度看，最终成品不仅要“能打开”，还要能通过代码、接口、脚本和产物四条证据链证明系统确实完成。